

Rocq projects

Formal Software Verification

Aarhus University

2026

This document lists the projects you may choose from for the Rocq project that closes the course. You may also invent a project yourself, or do a variation or an extension of one of the projects below; in that case, discuss it with the lecturer first.

Each project ends with an exercise listing possible extensions, of which you choose at least one. Complete the main body of the project before starting on an extension. A harder extension is necessary for a higher grade, and for a top grade you are expected to complete a very difficult one. In the report, follow the structure of the exercises, so that it is clear which extensions you have completed.

Each project is self-contained. This document gives every type and function you build on, and beyond them you need only Rocq's standard library — in particular `String.eqb`, `Nat.eqb` and `Nat.leb` for decidable equality and comparison, and `List` where a project uses lists. Identifiers are strings, as in the `Maps` chapter of *Logical Foundations*. Your development should need no other file: start from an empty `.v` file and the `Require Import` lines the project needs.

1 Project requirements

1.1 The Rocq development

Your development is a single Rocq file with no dependencies beyond the standard library, and every member of a group submits the same file, with the authors named in a comment at the top.

Structure it so that it is easy to follow: sensible and consistent names, judicious comments, unit tests or QuickChick properties for your functions, and `Section` and `Module` where they help. A long proof usually wants breaking into lemmas, a generalised statement is often easier to prove than the one you started with, and a repetitive proof is a candidate for Ltac.

Use the standard library wherever it helps, unless it trivialises the project: a sorting project that calls the library's sorting function, or a search tree project that instantiates its finite maps, has nothing left to prove. Ask if you are unsure. If your development involves numbers in a non-trivial way, expect to `Require Import Arith`.

1.2 The report

You may work in groups; a group writes one report together, and a bigger effort is expected from it, which usually means completing the very difficult extensions. Each of you remains responsible for all of the code and the report. **The oral exam also tests your general knowledge of the course material, including the chapters you did not hand in.**

Ten pages is an upper bound and shorter is fine. The report should describe the project in enough detail that a competent reader could reproduce your results without reference to your code, presenting the key ideas and proofs in mathematical language rather than in the language of Rocq tactics.

Introduction. What your key results are, what they mean, and how they fit into the bigger picture, with citations; you may need the literature for this. For a project on normalisation of the simply typed λ -calculus it would say what β -reduction is, what strong normalisation means, that not all terms have it, and that typing restores it.

Main results. One or more sections explaining the development and following its structure. Present everything mathematically where you can: typing rules belong in a deductive system, not as a Rocq inductive predicate. Leave out the routine steps of proofs, but give enough that a competent reader can check them.

Conclusions. The results, the insights behind them, your *acknowledgements* (see below), and a bibliography if you cite literature.

1.3 Help, collaboration and AI

Teamwork is encouraged, but it means pair programming: both of you at the same development, taking turns at the keyboard and discussing every step. It does not mean one of you doing most of the work, nor dividing the exercises between you.

The project is where you show what you have understood, and the oral exam starts from it. Help is allowed. Asking a classmate or an AI assistant to explain an error message, to suggest which lemma might apply, or to point you at the right chapter is fine, and so is working through the mathematics with your group. What is not fine is handing in work you cannot account for. A development or a report that rests on a team mate, on an AI assistant, or on a solution found online or borrowed from another group is cheating. Your knowledge of the project will be tested at the exam. Acknowledge everyone and everything that helped you, AI assistants included, and in what capacity.

2 SAT solver

In this project you implement a Boolean satisfiability (SAT) solver in Rocq and prove your implementation sound.

Boolean formulas are given by the following grammar:

$$p, q ::= x \mid \text{true} \mid \text{false} \mid p \wedge q \mid p \vee q \mid p \rightarrow q \mid \neg p.$$

Here x ranges over an infinite set of propositional variables. Examples of formulas are $(x \vee \neg y) \wedge (\neg x \vee y)$, $y \rightarrow (x \vee y)$ and $x \wedge \neg x \wedge \text{true}$.

A Boolean formula p is *satisfiable* in case its truth table contains at least one row whose outcome is 1. Of the three formulas above, the first two are satisfiable and the third is not.

Exercise 2.1 Define an inductive type *form* that represents Boolean formulas. Start by completing the following definition:

```
Inductive form :=
  | var : string -> form
  (* fill out *).
```

Exercise 2.2 Give Rocq definitions of type *form* corresponding to

$$(x \vee \neg y) \wedge (\neg x \vee y), \quad \neg y \rightarrow (x \vee y), \quad x \wedge \neg x \wedge \text{true}.$$

In order to define satisfiability we introduce the notion of a *valuation*, a function that assigns true or false to each propositional variable:

```
Definition valuation := string -> bool.
Definition empty_valuation : valuation := fun x => false.
Definition override (V : valuation) (x : string) (b : bool) : valuation :=
  fun y => if String.eqb x y then b else V y.
```

Exercise 2.3 Define the interpretation of a formula, using the truth-table semantics, by filling out the following definition:

```
Fixpoint interp (V : valuation) (p : form) : bool :=
  (* fill out *).
```

A formula is satisfiable if there exists a valuation that makes it true:

```
Definition satisfiable (p : form) : Prop :=
  exists V : valuation, interp V p = true.
```

Exercise 2.4 Prove in Rocq that $(x \vee \neg y) \wedge (\neg x \vee y)$ and $\neg y \rightarrow (x \vee y)$ are satisfiable. Create two lemmas of the shape

```
Lemma test1 : satisfiable (*formula 1*).
Lemma test2 : satisfiable (*formula 2*).
```

Exercise 2.5 Define a function that, given a formula, computes a valuation in which the formula is true. Implement it by enumerating all possible valuations, that is, by generating its truth table.

```
Definition find_valuation (p : form) : option valuation :=
  (* fill out *).
```

find_valuation should yield *None* in case no such valuation exists, that is, in case the formula is unsatisfiable.

We can now define the solver:

```
Definition solver (p : form) : bool :=
  match find_valuation p with
  | Some _ => true
  | None => false
```

end.

Exercise 2.6 Explain the difference between *satisfiable* and *solver*.

Exercise 2.7 Write 2 positive and 2 negative tests of the solver and prove them using the *reflexivity* tactic.

Exercise 2.8 Prove that *solver* is sound:

Lemma solver_sound : forall p, solver p = true -> satisfiable p.

Exercise 2.9 Extend your SAT solver in one of the following ways.

- Extend your development to three-valued logic (**moderate**). Three-valued logic has three truth values: true, false and some indeterminate third value. See https://en.wikipedia.org/wiki/Three-valued_logic.
- Write an optimiser that simplifies a Boolean formula using the laws $x \wedge \text{true} = x$, $\text{true} \wedge x = x$, $x \wedge \text{false} = \text{false}$, $\text{false} \wedge x = \text{false}$, $x \vee \text{true} = \text{true}$, $\text{true} \vee x = \text{true}$, $x \vee \text{false} = x$ and $\text{false} \vee x = x$, and prove it correct (**moderate**). Incorporate the optimiser into *solver* and its correctness proof.
- Write a converter that transforms Boolean formulas into negation normal form and prove it correct (**difficult**). A formula is in negation normal form if $\neg$ is applied only to variables. Incorporate it into *solver* and its correctness proof.
- Write a converter that transforms Boolean formulas into conjunctive normal form and prove it correct (**very difficult**). See https://en.wikipedia.org/wiki/Conjunctive_normal_form. Incorporate it into *solver* and its correctness proof.
- Prove completeness of your solver (**very difficult**):

Lemma solver_complete : forall p, satisfiable p -> solver p = true.

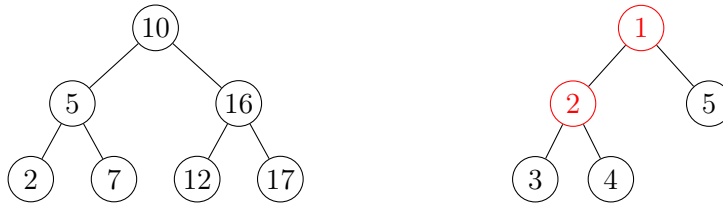
3 Binary search trees

In this project you implement operations on binary search trees in Rocq and prove the properties that matter of the implementation.

A binary search tree is a data structure that stores a finite set of natural numbers. Binary trees are defined in Rocq as follows:

```
Inductive tree :=
| leaf : tree
| node : tree -> nat -> tree -> tree.
```

A binary tree satisfies the *binary search tree property* if the value in each node is greater (but not equal) than all values in the left sub-tree, and smaller (but not equal) than all values in the right sub-tree. The tree on the left below satisfies the property; the one on the right does not, because a node numbered 2 occurs as a left sub-tree of a node numbered 1 (leaves are omitted).



Exercise 3.1 Give Rocq definitions of type `tree` corresponding to the two trees above.

Exercise 3.2 Define an inductively defined proposition that describes whether a binary tree satisfies the binary search tree property, by completing the following definition:

```
Inductive sorted : tree -> Prop :=
| leaf_sorted : sorted leaf
(* fill out *).
```

To define it you may want helpers `greater : nat -> tree -> Prop` and `smaller : nat -> tree -> Prop`, or `all : (nat -> Prop) -> tree -> Prop`, as inductively defined propositions.

Exercise 3.3 Prove that the first tree you defined satisfies the binary search tree property and that the second does not. You may want to use the composition `; of tactics` and the `repeat` tactical.

Exercise 3.4 Define a function that describes whether an element is contained in a binary search tree:

```
Fixpoint elem_of (x : nat) (t : tree) : bool :=
(* fill out *).
```

For an efficient implementation, do not traverse the whole tree: use the binary search tree property.

Exercise 3.5 Write 2 positive and 2 negative tests of `elem_of` and prove them using the `reflexivity` tactic.

The last part of this project is to define a function that inserts a natural number into a binary search tree. You have to prove that the resulting tree is indeed a binary search tree, and that insertion is correct.

Exercise 3.6 Define a function that inserts a natural number into a binary search tree:

```
Fixpoint insert (x : nat) (t : tree) : tree :=
(* fill out *).
```

If `x` is already in the tree, it should return the original tree.

Exercise 3.7 Write 2 tests of `insert` and prove them using `reflexivity`.

Exercise 3.8 *Prove that `insert` preserves the binary search tree property:*

Lemma `insert_sorted` : `forall t x, sorted t -> sorted (insert x t)`.

Exercise 3.9 *Prove that `insert` is correct:*

Lemma `insert_correct` : `forall t x y,`
`sorted t ->`
`elem_of y (insert x t) = orb (elem_of y t) (Nat.eqb x y)`.

Exercise 3.10 *Extend your development on binary search trees with a feature of your choice. For example:*

- A converter from lists to binary search trees and back. Prove a useful property about the interaction between these functions (**easy**).
- A delete function. Prove that it preserves the binary search tree property and prove a property about its interaction with `elem_of` (**moderate**).
- Self-balancing search trees, such as red-black trees or AVL trees (**very difficult**). See https://en.wikipedia.org/wiki/Self-balancing_binary_search_tree.

4 Arithmetic expression decompiler

In this project you formalise a compiler and a decompiler for an arithmetical expression language, prove the compiler correct, and prove the decompiler an inverse of it.

We consider the following expression language:

```
Inductive unop  := OSucc | OPred.  
Inductive binop := OPlus | OMult.  
Inductive exp  :=  
| ENat : nat -> exp  
| EUnOp : unop -> exp -> exp  
| EBinOp : binop -> exp -> exp -> exp.
```

ENat denotes a constant, OSucc computes the successor and OPred the predecessor (defined to be 0 in case the argument is 0), and OPlus and OMult perform addition and multiplication as usual.

Exercise 4.1 Define a function *eval* that gives a semantics to expressions:

```
Fixpoint eval (e : exp) : nat := (* fill out *).
```

Exercise 4.2 Write 2 tests of *eval* and prove them using *reflexivity*.

The first part of this project is to define a compiler from arithmetical expressions to a stack machine. The instructions of the stack machine are in reverse Polish notation:

```
Inductive instruction :=  
| IPush : nat -> instruction  
| IUnOp : unop -> instruction  
| IBinOp : binop -> instruction.
```

Exercise 4.3 Implement a compiler:

```
Fixpoint compile (e : exp) : list instruction := (* fill out *).
```

Exercise 4.4 Implement a virtual machine for stack instructions:

```
Definition vm : list instruction -> option nat := (* fill out *).
```

The virtual machine should yield *None* in case of stack underflow. For example, *vm [IBinOp OPlus; IPush 10] = None*.

Exercise 4.5 Write 2 positive and 2 negative tests of the virtual machine and prove them using *reflexivity*.

Exercise 4.6 Prove correctness of your compiler:

```
Lemma vm_correct : forall e, vm (compile e) = Some (eval e).
```

The next part is to define a decompiler, which translates a list of machine instructions into a corresponding arithmetical expression. Its implementation is very similar to that of the virtual machine; instead of evaluating a list of instructions to a value, you evaluate it to an expression.

Exercise 4.7 Define a decompiler:

```
Definition decompile : list instruction -> option exp := (* fill out *).
```

It should yield *None* in case the list of instructions is ill-formed. For example, *decompile [IBinOp OPlus; IPush 10] = None*.

Exercise 4.8 Prove that the decompiler is a left inverse of the compiler:

```
Lemma decompile_correct : forall e, decompile (compile e) = Some e.
```

Exercise 4.9 Extend your project in one of the following ways.

- Extend the language with an operator that may fail, for example division, which may fail in case of division by zero (*moderate*). You have to extend the syntax and semantics of the

expression and machine language, as well as your proofs.

- *Extend the language with a non-deterministic random number generator (**difficult**). You have to extend the syntax and semantics of the expression and machine language, as well as your proofs.*
- *Prove that the decompiler is a right inverse of the compiler (**difficult**):*

```
Lemma compile_decompile : forall ins e,  
  decompile ins = Some e -> compile e = ins.
```

- *The decompiler and the virtual machine, as well as their correctness proofs, are very similar. Find an abstraction that factors out the similarity (**very difficult**).*

5 Confluence of the SKI calculus

In this project you prove that the SKI calculus is confluent.

The SKI calculus has terms built from the combinators S , K , I and application, and the reduction rules

$$I x \rightarrow x, \quad K x y \rightarrow x, \quad S x y z \rightarrow x z (y z),$$

closed under application on both sides. Confluence is the following statement.

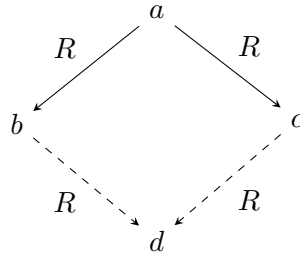
Theorem. For all SKI terms x, y, z , if $x \rightarrow^* y$ and $x \rightarrow^* z$, then there exists w such that $y \rightarrow^* w$ and $z \rightarrow^* w$.

In this project you prove this theorem formally in Rocq. To do so, you will have to formalise the terms of SKI with an inductive type, formalise the reduction relation with an inductive predicate, and formulate and prove confluence.

Suggested approach

Confluence can be characterised as the reflexive-transitive closure of the reduction relation ($\rightarrow^*$) having the following strong diamond property.

Definition (strong diamond property). A relation $R \subseteq A \times A$ has the *strong diamond property* if for all $a, b, c \in A$ with $a R b$ and $a R c$ there exists $d \in A$ with $b R d$ and $c R d$.



The following general result gives a way of proving that the reflexive-transitive closure of a relation has the strong diamond property.

Lemma. Suppose that a relation R has the strong diamond property. Then its reflexive-transitive closure R^* has the strong diamond property.

Unfortunately $\rightarrow$ itself does not have the strong diamond property. The idea instead is to define a new relation $\xrightarrow{M}$ with the following properties.

Lemma. The relation $\xrightarrow{M}$ has the strong diamond property.

Lemma. For all terms x, y : $x \rightarrow^* y$ if and only if $x \xrightarrow{M^*} y$.

The theorem then follows as a corollary of these three lemmas.

A challenge of this project is to find an appropriate definition of $\xrightarrow{M}$. It should allow multiple simultaneous reductions in a term, so that for instance $SKI(IK) \xrightarrow{M} KK(IK)$ in one step, by reducing the outer S and the inner IK at the same time.

Proving the strong diamond property for $\xrightarrow{M}$ directly means analysing two derivations at once, which is painful. There is a way around it worth knowing, the method of *complete developments*.

Define a *function* t^* that contracts every redex of t at once — it is an ordinary **Fixpoint** on terms, with one case per redex shape — and prove the single statement

$$\text{if } t \xrightarrow{M} u \text{ then } u \xrightarrow{M} t^*.$$

The strong diamond property for $\xrightarrow{M}$ follows at once, taking t^* as the common reduct d : whatever two terms t reduces to in parallel, both reduce in parallel to t^* . The statement above is proved by induction on the derivation of $t \xrightarrow{M} u$, and the work in it is in the application case, where you have to ask whether the application is itself a redex. Small inversion lemmas — such as: if $K \xrightarrow{M} v$ then $v = K$, and a parallel reduct of $K x$ is of the form $K x'$ with $x \xrightarrow{M} x'$ — keep that case readable.

Exercise 5.1 *Extend the project in one of the following ways.*

- *Extend the calculus with other combinators such as B , C , W (**easy**).*
- *Can the calculus be extended in a modular way? That is, is there a property that can be checked on each combinator or rule separately that guarantees that the combined system is confluent (**difficult**)?*
- *Show the confluence of the λ -calculus (**very difficult**).*

6 2–3 trees

In this project you implement and verify a 2–3 tree.

A 2–3 tree is a form of self-balancing search tree (a special case of a B-tree) in which each node has either two or three children; at a leaf, these children are the empty tree. A 2-node has two children and one key: all keys in the left child are lower than the key, and all keys in the right child are higher. A 3-node has three children and two keys: all keys in the first child are lower than the first key; all keys in the second child are between the first and second keys; and all keys in the third child are above the second key.

To determine whether a key is in the tree, use the search tree property to walk the one path that is guaranteed to contain the key if it is in the tree at all. For simplicity we do not view the tree as mapping keys to values; we treat it as representing a finite set of keys.

To insert a key, find the leaf where it would belong if it were in the tree. (If it is already in the tree, nothing need be done.) Add the key to that leaf. If the leaf is a 2-node, it becomes a 3-node. If it is a 3-node, it is split into two 2-nodes holding the lowest and the greatest key, and the middle key is promoted to the next level in the tree. At the upper level this means replacing the child with a key and two children, which will either turn a 2-node into a 3-node or split a 3-node. This proceeds up the tree. If the root is split, it is replaced by a new 2-node. Figure 1 illustrates the process.

For this project, you should

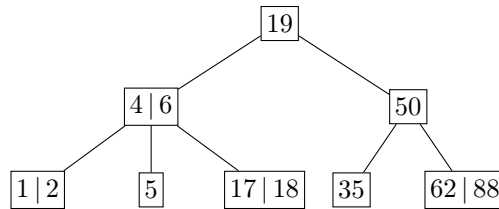
- define an inductive data type to represent a 2–3 tree;
- implement the search and insert functions on 2–3 trees;
- test these functions on suitable inputs;
- define a predicate that expresses when a key is in a 2–3 tree;
- define well-formedness predicates on 2–3 trees that express the invariants that they are balanced and represent search trees;
- show that search and insert behave correctly when the invariants hold;
- show that insert preserves the invariants.

Suggested approach

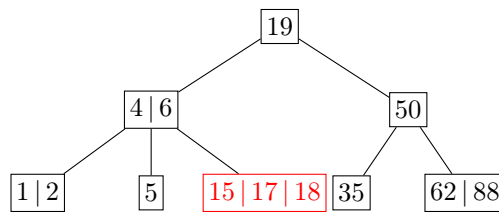
Defining the insert function correctly is the hard part, and it may need auxiliary inductive types and functions of its own; the comparison operator `?=` of the standard library helps. The proofs repeat many similar cases, so automate them with Ltac.

Exercise 6.1 *Extend the project in one of the following ways.*

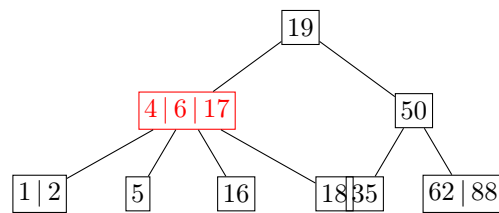
- Store values in the tree associated with keys, so that the tree represents a finite map rather than a finite set (**moderate**).
- Implement and verify deletion from the tree (**very difficult**). Look up how B-trees delete and adapt it.
- Generalise the tree and the proofs to any key type with a comparison operation, using a type class that carries the laws the proofs need (**moderate**).



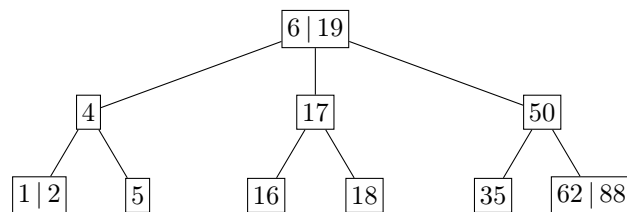
(a) the tree before insertion



(b) 15 is added to the leaf it belongs in, which overflows



(c) the middle key is promoted, and the parent overflows in turn



(d) the root splits, and the tree grows one level

Figure 1: Inserting the value 15 into a 2–3 tree.

Acknowledgements

The SAT solver, the binary search trees and the arithmetic expression decompiler are projects by **Robbert Krebbers**.

The project requirements, the confluence of the SKI calculus and the 2–3 trees are by **Thomas Dinsdale-Young**, who also drew the 2–3 tree insertion figure.